

CUSTOMER SHOPPING BEHAVIOR ANALYSIS

BY SPARSH VERMA

1. Project Overview

This project analyzes customer shopping behavior using transactional data from 3,900 purchases across various product categories. The goal is to uncover insights into spending patterns, customer segments, product preferences, and subscription behavior to guide strategic business decisions.

2. Dataset Summary

- Rows: 3,900
- Columns: 18
- Key Features:
 - Customer demographics (Age, Gender, Location, Subscription Status)
 - Purchase details (Item Purchased, Category, Purchase Amount, Season, Size, Color)
 - Shopping behavior (Discount Applied, Promo Code Used, Previous Purchases, Frequency of Purchases, Review Rating, Shipping Type)
- Missing Data: 37 values in Review Rating column

3. Exploratory Data Analysis using Python

We began with data preparation and cleaning in Python:

- **Data Loading** – Imported the dataset using pandas.
- **Initial Exploration:** Used `df.info()` to check structure and `.describe()` for summary statistics.

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Shipping Type	Discount Applied
count	3900.000000	3900.000000	3900	3900	3900	3900.000000	3900	3900	3900	3900	3863.000000	3900	3900	
unique	NaN	NaN	2	25	4	NaN	50	4	25	4	NaN	2	6	
top	NaN	NaN	Male	Blouse	Clothing	NaN	Montana	M	Olive	Spring	NaN	No	Free Shipping	
freq	NaN	NaN	2652	171	1737	NaN	96	1755	177	999	NaN	2847	675	
mean	1950.500000	44.068462	NaN	NaN	NaN	59.764359	NaN	NaN	NaN	NaN	3.750065	NaN	NaN	
std	1125.977353	15.207589	NaN	NaN	NaN	23.685392	NaN	NaN	NaN	NaN	0.716983	NaN	NaN	
min	1.000000	18.000000	NaN	NaN	NaN	20.000000	NaN	NaN	NaN	NaN	2.500000	NaN	NaN	
25%	975.750000	31.000000	NaN	NaN	NaN	39.000000	NaN	NaN	NaN	NaN	3.100000	NaN	NaN	
50%	1950.500000	44.000000	NaN	NaN	NaN	60.000000	NaN	NaN	NaN	NaN	3.800000	NaN	NaN	
75%	2925.250000	57.000000	NaN	NaN	NaN	81.000000	NaN	NaN	NaN	NaN	4.400000	NaN	NaN	
max	3900.000000	70.000000	NaN	NaN	NaN	100.000000	NaN	NaN	NaN	NaN	5.000000	NaN	NaN	

- **Missing Data Handling:** Checked for null values and imputed missing values in the **Review Rating** column using the median rating of each product category.

```
df["Review Rating"] = df.groupby("Category")["Review Rating"].transform(lambda x: x.fillna(x.median()))
```

- **Column Standardization:** Renamed columns to **snake case** for better readability and documentation.

```
df.columns = df.columns.str.lower()
df.columns = df.columns.str.replace(' ', '_')
df = df.rename(columns={'purchase_amount_(usd)': 'purchase_amount'})
```

- **Feature Engineering:**

- Created **age_group** column by binning customer ages based on pre-defined age ranges.

```
# create age-group column
df['age_group'] = pd.cut(
    df['age'],
    bins = [df['age'].min(), 25, 40, 60, df['age'].max()],
    labels = ["Young Adult", "Adult", "Middle-aged", "Senior"],
    include_lowest = True
)
```

Here we used `pd.cut()` instead of `pd.qcut()` because:

- The former reflects real population distribution based on pre-defined ranges.
- The latter can introduce statistical bias.

- Created **purchase_frequency_days** column from purchase data.

```
# create purchase_frequency_days column

freq_mapping = {
    'Fortnightly': 14,
    'Weekly': 7,
    'Annually': 365,
    'Quarterly': 90,
    'Bi-Weekly': 14,
    'Monthly': 30,
    'Every 3 Months': 90,
}

df['purchase_frequency_days'] = df['frequency_of_purchases'].map(freq_mapping)
```

- **Data Consistency Check:** Verified if `discount_applied` and `promo_code_used` were redundant; dropped `promo_code_used`.

```
(df['discount_applied'] == df['promo_code_used']).all()
```

```
np.True_
```

```
df = df.drop('promo_code_used', axis=1)
df.columns
```

- **Database Integration:** Connected Python script to PostgreSQL and loaded the cleaned DataFrame into the database for SQL analysis. (PostgreSQL)

```
pip install psycopg2-binary sqlalchemy
```

```
# Step 1: Connect to PostgreSQL
# Replace placeholders with your actual details
from sqlalchemy import create_engine
username = "postgres" # default user
password = "2305" # password set during installation
host = "localhost" # if running locally
port = "2305" #Postgre port set during installation
database = "customer_behaviour" # the database created in pgAdmin

engine = create_engine(f"postgresql+psycopg2://{username}:{password}@{host}:{port}/{database}")

# Step 2: Load dataframe into SQL
table_name = "customer" # choose any name
df.to_sql(table_name, engine, if_exists="replace", index=False)

print(f"Data successfully loaded into table '{table_name}' in database {database}.")

Data successfully loaded into table 'customer' in database customer_behaviour'.
```

4. Data Analysis using SQL (PostgreSQL)

We performed structured analysis in PostgreSQL to answer key business questions:

1. **Revenue by Gender** - Compared total revenue generated by male vs. female customers.

Query –

```
select gender, SUM(purchase_amount) as revenue
from customer
group by gender
```

Output –

	gender text	revenue numeric
1	Female	75191
2	Male	157890

2. **High-Spending Discount Users** - Identified customers who used discounts but still spent above the average purchase amount and the total number of customers that satisfy this condition out of total customers

Query –

```
select customer_id, purchase_amount
from customer
where discount_applied = 'Yes' and purchase_amount >= (select AVG(purchase_amount) from customer)

--No. of customers that satisfy Q2.
select
count(*) filter (where discount_applied = 'Yes'
and purchase_amount >= (select avg(purchase_amount) from customer)) as qualifying_customers,
count(*) as total_customers
from customer
```

Output –

	customer_id bigint	purchase_amount bigint
1	2	64
2	3	73
3	4	90
4	7	85
5	9	97
6	12	68
7	13	72
8	16	81
9	20	90
Total rows: 839		Query complete 00:00:00.142

	qualifying_customers bigint	total_customers bigint
1	839	3900

3. **Top 5 products by rating** - Identified products with the highest average review ratings

Query –

```
select item_purchased, round(avg(review_rating::numeric), 2) as "Avg Product Rating"
from customer
group by item_purchased
order by avg(review_rating) desc
limit 5;
```

Output –

	item_purchased text	Avg Product Rating numeric
1	Gloves	3.86
2	Sandals	3.84
3	Boots	3.82
4	Hat	3.80
5	Skirt	3.78

Insight –

Here we explicitly changed the type of “review_rating” as while connecting the DB from python to SQL, the system has typecasted this column as double precision, which can’t be rounded off in SQL.

4. **Shipping Type Comparison** – Compared average purchased amounts between Standard and Express shipping.

Query –

```
select shipping_type, round(avg(purchase_amount), 2)
from customer
where shipping_type in ('Standard','Express')
group by shipping_type
```

Output –

	shipping_type text	round numeric
1	Standard	58.46
2	Express	60.48

5. **Subscribers vs. Non-subscribers** – Compared average spend and total revenue across subscription status.

Query –

```
select subscription_status,
count(customer_id) as "Total Customers",
sum(purchase_amount) as "Total Revenue",
round(avg(purchase_amount), 2) as "Average Spend"
from customer
group by subscription_status
```

Output –

	subscription_status text	Total Customers bigint	Total Revenue numeric	Average Spend numeric
1	No	2847	170436	59.87
2	Yes	1053	62645	59.49

6. **Discount-dependent products** – Identified 5 products with the highest percentage of discounted purchases.

Query –

```
select item_purchased,
round(100 * sum(case when discount_applied = 'Yes' then 1 else 0 end)/count(*),2) as discount_rate
from customer
group by item_purchased
order by discount_rate desc
limit 5;
```

Output –

	item_purchased text	discount_rate numeric
1	Hat	50.00
2	Sneakers	49.00
3	Coat	49.00
4	Sweater	48.00
5	Pants	47.00

7. **Customer Segmentation** – Classified customers into New, Returning, and Loyal segments based on purchase history.

Query –

```
with customer_type as (
select customer_id, previous_purchases,
case
when previous_purchases = 1 then 'New'
when previous_purchases between 2 and 10 then 'Returning'
else 'Loyal'
end as customer_segment
from customer
)

select customer_segment, count(*) as "No of Customers"
from customer_type
group by customer_segment
```

Output –

	customer_segment text	No of Customers bigint
1	Loyal	3116
2	New	83
3	Returning	701

Insight – In this query we used CTE (common table expression), it is used to store temporary result and exists for the duration of the query. It's used for complex/advanced queries.

(with customer_type as (. . .))

8. **Top 3 products per category** – Listed the most purchased products within each category.

Query –

```
with item_counts as (  
  select category, item_purchased,  
  count(customer_id) as total_orders,  
  row_number() over(partition by category order by count(customer_id) desc) as item_rank  
  from customer  
  group by category, item_purchased  
)  
select item_rank, category, item_purchased, total_orders  
from item_counts  
where item_rank <= 3;
```

Output –

	item_rank bigint	category text	item_purchased text	total_orders bigint
1	1	Accessori...	Jewelry	171
2	2	Accessori...	Sunglasses	161
3	3	Accessori...	Belt	161
4	1	Clothing	Blouse	171
5	2	Clothing	Pants	171
6	3	Clothing	Shirt	169
7	1	Footwear	Sandals	160
8	2	Footwear	Shoes	150
9	3	Footwear	Sneakers	145
10	1	Outerwear	Jacket	163
11	2	Outerwear	Coat	161

Insight –

A. Here we have used a Window Function instead of a group by, because the former assigns rank to existing rows whereas the latter collapses rows and creates groups.

B. Here we used `row_number()` instead of `row()` and `dense_row()`: It gives us a unique rank even for the observations that have the same total whereas `row()` gives same rank to same totals and skips the number of next ranks and `dense_row()` gives the same rank to same totals but doesn't skip the next rank.

9. **Repeat Buyers & Subscriptions** – Checked whether customers with >5 purchases are more likely to subscribe.

Query –

```
select subscription_status,  
count(customer_id) as repeat_buyers  
from customer  
where previous_purchases > 5  
group by subscription_status
```

Output –

	subscription_status text	repeat_buyers bigint
1	No	2518
2	Yes	958

Bonus Question –

Bayes Theorem: Find the probability that a random customer is a subscriber given that he/she is a repeat buyer.

Query –

```
with probabilities as (  
  select  
    count(*) filter (  
      where subscription_status = 'Yes')::numeric/count(*) as p_s,  
    count(*) filter (  
      where previous_purchases > 5)::numeric/count(*) as p_r,  
    count(*) filter (  
      where subscription_status = 'Yes' and previous_purchases > 5)::numeric/count(*) as p_r_given_s  
  from customer  
)  
select  
  round(p_r_given_s,3) as "P(Repeat Buyer | Subscriber)", round(p_s,3) as "P(Subscriber)",  
  round(p_r,3) as "P(Repeat Buyer)",  
  round((p_r_given_s * p_s/p_r),3) as "P(Subscriber | Repeat Buyer)"  
from probabilities;
```

Output –

	P(Repeat Buyer Subscriber) numeric	P(Subscriber) numeric	P(Repeat Buyer) numeric	P(Subscriber Repeat Buyer) numeric
1	0.246	0.270	0.891	0.074

10. **Revenue by age group** – Calculated total revenue contribution of each age group.

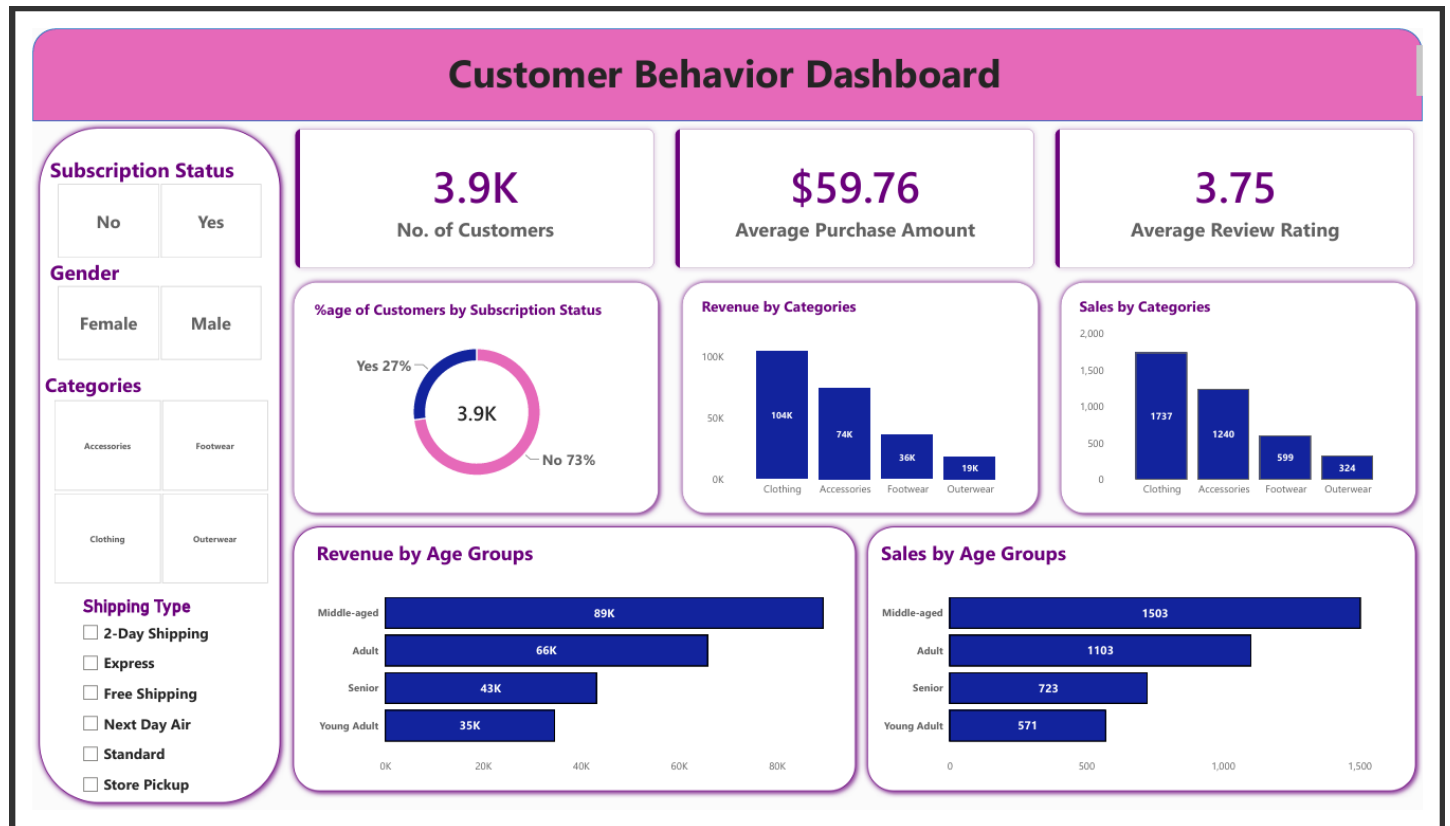
Query –

```
select age_group, sum(purchase_amount) as total_rev  
from customer  
group by age_group  
order by total_rev desc |
```

Output –

	age_group text	total_rev numeric
1	Middle-aged	89445
2	Adult	65842
3	Senior	43164
4	Young Adult	34630

5. Power BI Dashboard



6. Challenges identified during Data Analysis

- Massive gender revenue imbalance**

Male customers generated \$157,890 vs. just \$75,191 from female customers - a 2.1x gap. For a retail business selling across Clothing, Footwear, Accessories, and Outerwear, this suggests either a merchandising/marketing skew toward men or a genuine failure to engage female shoppers, who represent a huge, untapped pool.
- Subscription program isn't paying for itself**

Subscribers spend *less* on average (\$59.49) than non-subscribers (\$59.87), and only 1,053 of 3,900 customers (27%) are subscribed. Worse, the Bayes analysis shows $P(\text{Subscriber} \mid \text{Repeat Buyer})$ is only 0.074 - meaning even loyal, repeat customers rarely convert to subscription. The subscription offer clearly isn't compelling enough to drive either loyalty or higher expenditure.
- Customer retention is critically weak**

The segmentation query shows only 701 "Returning" customers and a mere 83 "New" customers, against 3,116 already "Loyal" (>10 purchases). This distribution looks inverted - a healthy funnel needs a strong New → Returning pipeline feeding the Loyal tier. With only 83 new customers in the dataset, acquisition and early-stage retention are bottlenecked.

4. Heavy discount dependency on core products

Hat (50%), Sneakers (49%), Coat (49%), Sweater (48%), and Pants (47%) are being discounted on nearly half of all transactions. Combined with 839 of 3,900 customers (21.5%) being "high-spending discount users" who'd likely buy anyway, the business is leaking margin on products that may not need the incentive.

5. Revenue heavily concentrated in older age segments

Middle-aged and Adult segments drive $\$89,445 + \$65,842 = \$155,287$ (72% of total revenue), while Young Adults contribute only $\$34,630$ (16%). This signals a weak pipeline of younger customers - a demographic risk for long-term growth as this cohort ages past the buying window.

7. Business Recommendations

1. Rebuild the subscription value proposition

Redesign perks (free express shipping, early access, subscriber-only discounts) targeted at the 3,116 Loyal customers who aren't yet subscribed.

Target: Lift subscription rate from 27% → 40% and close the average-spend gap (subscribers should out-spend non-subscribers by approx. 10%, i.e., \$65 vs. \$59.87)

Impact: At 40% of 3,900 customers × \$65 avg spend, subscription revenue could rise from \$62,645 to roughly **\$101,400 (+62%)**

2. Launch a female-focused acquisition and merchandising push

Investigate why female revenue trails male by more than 2x — likely category assortment, marketing spend allocation, or pricing. Run targeted campaigns and category expansion.

Target: Close the gap from 2.1x to 1.5x

Impact: Female revenue moving from \$75,191 to **\$105,000 (+40%)**, adding approx. \$30K in incremental revenue

3. Fix the acquisition funnel — New customer count is the biggest red flag

Only 83 "New" customers against 3,116 "Loyal" is statistically unusual and suggests either data skew or a genuine acquisition problem. Invest in referral programs, first-purchase discounts, and awareness campaigns.

Target: Grow New customer share from 2% to 10% of the base (approx. 390 customers) to properly feed the Returning → Loyal pipeline

Impact: A healthier funnel typically improves 12-month retention by **15-20%**, directly protecting the \$170K+ revenue currently concentrated in the Loyal tier

4. Tighten discount policy on the top 5 over-discounted products

Reduce blanket discounting on Hat, Sneakers, Coat, Sweater, and Pants from 48% to approx. 25% of transactions, replacing broad discounts with targeted ones (e.g., only for price-sensitive segments identified via the 839 high-spending discount users).

Target: Recover margin without losing volume, since many of these buyers spend above average anyway

Impact: If half the discount frequency is recovered at full price, this could reclaim an estimated **8-12% margin** on these 5 product lines

5. Build a Young Adult acquisition and engagement strategy

Given Young Adults contribute only \$34,630 (16% of revenue) despite typically being high lifetime-value customers, launch social-first marketing, express/2-day shipping promos (their likely preference), and entry-level price points.

Target: Grow Young Adult revenue share from 16% to 22% of total

Impact: This would add roughly **\$32,000 in incremental revenue** without cannibalizing other segments